OceanBase × LangChain Community Series

Building Agentic RAG with LangChain & OceanBase

Episode 1 Understanding LangChain Agents

Haili Zhang · LangChain Ambassador


OCEANBASE

×


LangChain Community

Haili Zhang

LangChain Ambassador

GitHub	@webup
Twitter	@zhanghaili0610
Bilibili	沧海九粟

What We'll Cover Today

Three core topics for building LangChain agents

Construct Agents

`create_agent()`

Building agents with models and tools

Foundation

Tool Integration

`@tool`

Enabling agents to take actions

Capabilities

Execution

`invoke` & `stream`

Running then observing progress

Runtime

Let's start with the basics

Construct Agents

Building the foundation with create_agent()

What is an Agent?

Systems that reason, decide, and act autonomously

Language Models

- **Reasoning Engine**
  Makes decisions about what to do next
- **Natural Language Understanding**
  Interprets user requests and tool outputs
- **Plan & Execute**
  Determines the sequence of actions needed

Uses of Tools

- **Take Actions**
  Search, calculate, API calls, database queries
- **Return Observations**
  Feed results back to the model
- **Extend Capabilities**
  Connect to external systems and data

Execution Loop

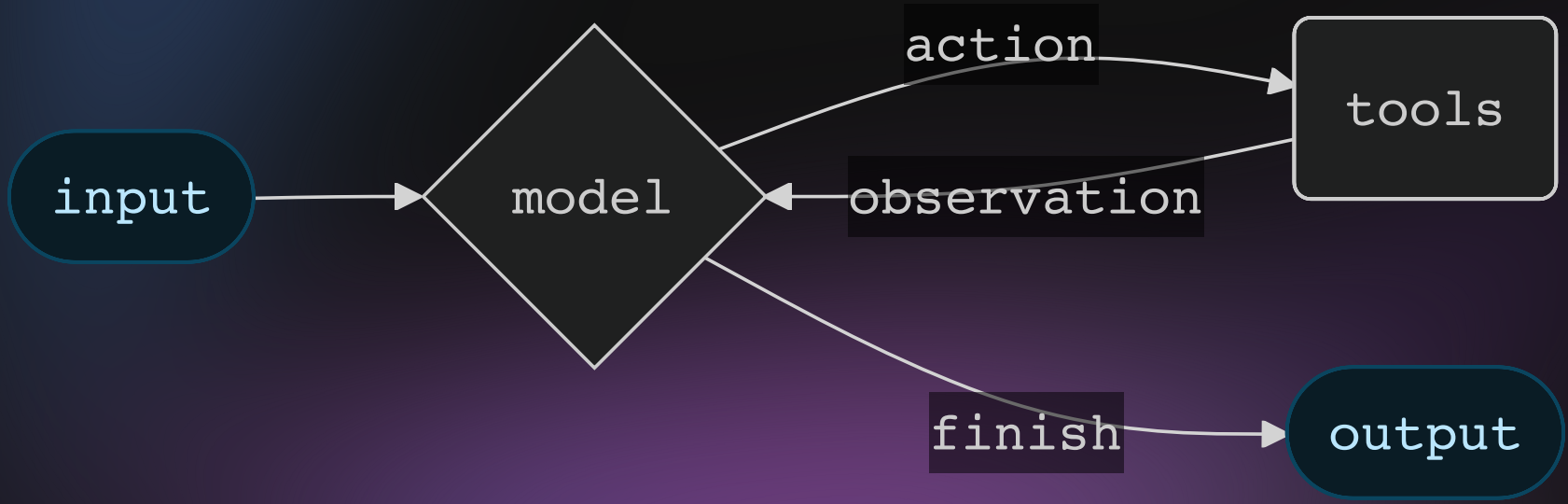
- **Iterative Process**
  Runs until goal is achieved
- **Dynamic Decisions**
  Adapts based on tool results
- **Stop Conditions**
  Final output or iteration limit reached

"An LLM Agent runs tools in a loop to achieve a goal"

An agent runs until a stop condition is met - i.e., when the model emits a final output or an iteration limit is reached.

The ReAct Loop

Reasoning + Acting in continuous cycles



arXiv ReAct: Synergizing Reasoning and Acting in Language Models

Models: Models

The reasoning engine of your agent

Static Model

```python
# String identifier
agent = create_agent("gpt-4o", tools=tools)

# Or direct instance
model = ChatOpenAI(model="gpt-4o")
agent = create_agent(model, tools=tools)
```

- Configured once
- Simple and straightforward

Dynamic Model

```python
@wrap_model_call
def route_model(request, handler):
    if request.state["messages"] > 10:
        request.model = advanced_model
    return handler(request)
```

- Selected at runtime
- Based on state/context

Use static for simplicity, dynamic for cost optimization or complex routing

Tools: Tools

Giving agents the ability to take actions

Defining Tools

```python
@tool
def search(query: str) → str:
    """Search for information."""
    return f"Results for: {query}"

@tool
def get_weather(location: str) → str:
    """Get weather info."""
    return f"Weather: {location}"
```

⚠ Error Handling

```python
@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"Error: {e}",
            tool_call_id=request.id
        )
```

What Agents Enable Beyond Simple Tool Binding

- Multiple sequential tool calls
- Parallel tool execution
- Dynamic tool selection
- Retry logic
- State persistence
- Error recovery

System Prompt

Shaping agent behavior and approach

Static System Prompt

```
agent = create_agent(
    model,
    tools,
    system_prompt=
        "You are a helpful assistant. "
        "Be concise and accurate."
)
```

- Simple string parameter
- Same for all interactions

Dynamic System Prompt

```
@dynamic_prompt
def user_role_prompt(request):
    role = request.runtime.context \
        .get("user_role", "user")
    if role == "expert":
        return "Provide detailed ..."
    return "Explain simply ... "
```

- Based on runtime context
- Adapts to user needs

Memory: Short-term

Maintaining conversation context and custom state

Message History (Automatic)

Agents automatically maintain conversation history through message state

- Built-in message tracking
- Context preserved across turns
- No configuration needed

Custom State

```python
class CustomState(AgentState):
    user_preferences: dict

agent = create_agent(
    model, tools,
    state_schema=CustomState
)
```

- Track custom data
- Must be TypedDict (v1.0+)

For persistence across sessions, see Long-term Memory patterns

Production Architecture

Built on LangGraph for reliability and scalability

Graph-Based Runtime

Provides structure for complex agent workflows

- Nodes: steps (model, tools, middleware)
- Edges: connections between steps
- State: flows through the graph

Production Features

- Persistence across steps
- Error recovery & retry logic
- Streaming support
- Human-in-the-loop patterns

Manual Implementation

1-2 Days

Per complex workflow

- Custom orchestration logic
- Error handling edge cases
- Extensive testing needed

With LangChain Agents

1-2 Hours

Using create_agent()

- Pre-built orchestration
- Robust error handling
- Production-ready

Now let's integrate capabilities

Tool Use

Giving agents the power to take actions with the tool decorator

Tools: Enabling Agent Actions

Beyond simple model-tool binding

Defining Tools

```python
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

@tool
def get_weather(location: str) -> str:
    """Get weather info."""
    return f"Weather in {location}: "
           "Sunny, 72°F"

agent = create_agent(
    model,
    tools=[search, get_weather]
)
```

What Agents Enable

Beyond Simple Tool Binding

- Multiple sequential tool calls
- Parallel tool execution
- Dynamic tool selection

Built-in Capabilities

- Retry logic
- State persistence

Empty tool list creates an agent without tool-calling capabilities

Tool Error Handling

Graceful error management with middleware

Define Error Middleware

```python
from langchain.agents.middleware import (
    wrap_tool_call
)
from langchain_core.messages import (
    ToolMessage
)

@wrap_tool_call
def handle_tool_errors(request, handler):
    """Custom error handling."""
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"Error: {str(e)}",
            tool_call_id=request.tool_call["id"]
        )
```

Apply Middleware

```python
agent = create_agent(
    model="gpt-4o",
    tools=[search, get_weather],
    middleware=[handle_tool_errors]
)
```

Error Response

Returns custom ToolMessage:

```
ToolMessage(
  content="Error: division by zero",
  tool_call_id="..."
)
```

Tool Use in the ReAct Loop

Example Prompt: *"Find the most popular wireless headphones right now and check if they're in stock"*

Cycle 1: Search

Reasoning: "Popularity is time-sensitive, I need to search."

Acting: `search_products("wireless headphones")`

Tool Response:
```
Found 5 products. Top result: WH-1000XM5
```

Cycle 2: Verify Inventory

Reasoning: "Need to confirm availability for top item."

Acting: `check_inventory("WH-1000XM5")`

Tool Response:
```
Product WH-1000XM5: 10 units in stock
```

Final Answer

Reasoning: "I have the model and stock status. Ready to answer."

Response: "Found wireless headphones (WH-1000XM5) with 10 units in stock..."

The agent alternates between reasoning and acting until it can deliver a complete answer

Tool Categories for ReAct Agents

Common tool types for production systems

Search Tools

Web & API queries

Retriever Tools

Vector search RAG

Filesystem Tools

Read & write files

Multimodal Tools

Vision & audio

Best Practices

- Provide clear tool descriptions for better selection
- Implement proper error handling and retry logic
- Combine multiple tool types for complex workflows

Ready to put it all together

Execution

Running agents with invoke() & stream(), plus advanced patterns

Invocation: Running Your Agent

Executing agents by passing updates to their state

Basic Invocation

```python
# Invoke with new message
result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "What's the weather in SF?"
    }]
})

# Access final response
final_message = result["messages"][-1]
```

- Pass messages to agent state
- Blocks until execution finishes

Continuing Conversations

```python
# Continue existing conversation
result = agent.invoke({
    "messages": result["messages"] + [{
        "role": "user",
        "content": "How about tomorrow?"
    }]
})

# Agent has full context from previous turns
```

- Append to existing state
- Maintains conversation context

`invoke()` runs the complete agent loop and returns when a final answer is ready

Streaming: Streaming

Surface live feedback from agent runs to your application

Agent Progress
State updates after each step

LLM Tokens
Tokens as they're generated

Custom Updates
User-defined signals

Multiple Modes
Combine different types

Stream Mode: updates

Get state updates after each agent step

```
for chunk in agent.stream({
    "messages": [{"role": "user", "content": ""}]
}, stream_mode="updates"):
    # Each chunk is a state update
    if "messages" in chunk:
        print(f"Step: {chunk}")
```

Stream Mode: messages

Stream LLM tokens plus message metadata

```
for chunk in agent.stream({
    "messages": [ ... ]
}, stream_mode="messages"):
    # Tokens as they're generated
    if chunk.content:
        print(chunk.content, end="")
```

Combine modes: `stream_mode=["updates", "messages"]` for both agent steps and token streaming

Middleware: Middleware

Extensibility at every stage of execution

Before Model

Message trimming, context injection

After Model

Guardrails, content filtering

Tool Calls

Error handling, logging

Before Model: Message Trimming

```python
@before_model
def trim_messages(state, runtime):
    messages = state["messages"]
    if len(messages) > 10:
        return {"messages": messages[-10:]}

agent = create_agent( ... , middleware=[trim_messages])
```

After Model: Content Filter

```python
@after_model
def filter_data(state, runtime):
    last_msg = state["messages"][-1]
    if contains_unsafe(last_msg.content):
        last_msg.content = "[filtered]"
    return state

agent = create_agent( ... , middleware=[filter_data])
```

Middleware integrates seamlessly – Intercept data flow without changing core logic

Structured Output: Structured Output

Getting responses in specific formats

Define Schema

```python
from pydantic import BaseModel

class ContactInfo(BaseModel):
    name: str
    email: str
    phone: str
```

ToolStrategy

```python
agent = create_agent(
    model="gpt-4o-mini",
    response_format=
        ToolStrategy(
            ContactInfo
        )
)
```

- Uses artificial tool calling
- Works with any model

ProviderStrategy

```python
agent = create_agent(
    model="gpt-4o",
    response_format=
        ProviderStrategy(
            ContactInfo
        )
)
```

- Uses native structured output
- More reliable (OpenAI, etc.)

Result available in `result["structured_response"]`

What did we learn?

Key Insights

Essential principles for building production-ready LangChain agents

Key Takeaways

Building effective agents with LangChain

1 **Agents = Models + Tools + Loop**

Combine reasoning with actions for autonomous task solving

2 **Start simple, extend as needed**

Static models and basic tools → Dynamic routing and middleware

3 **Production-ready from the start**

Built on LangGraph with streaming, persistence, and error handling

4 **Extensible through middleware**

Customize behavior at every stage without changing core logic

Agent Development Speed

Time to production-ready agent systems

Native LangGraph

Define graph structure:	**1-2 hours**
Implement nodes/edges:	**2-3 hours**
State schema setup:	**30-60 min**
Testing & debugging:	**2-3 hours**

LangChain Agents

Graph auto-generated:	**0 min**
Pre-built patterns:	**~30 min**
State managed:	**automatic**
Testing & debugging:	**1-2 hours**

Development Time Comparison

Native LangGraph

6-9 hours

Full control & flexibility

LangChain Agents

2-3 hours → **~60% faster**

Less boilerplate, faster prototyping

Development Environment Setup

Modern Python tooling for LangChain agents

```toml
# pyproject.toml
[project]
name = "my-agent"
requires-python = "≥3.11,<4.0"
dependencies = [
  "langchain≥1.0.2,<2.0.0",
  "langchain-core≥1.0.0,<2.0.0",
  "langgraph≥0.2.0,<1.0.0",
  "langchain-anthropic≥1.0.0,<2.0.0",
]

[dependency-groups]
dev = ["langchain-openai", "ruff≥0.12.2"]
test = ["pytest", "pytest-cov", "mypy≥1.18.1"]
```

Development Workflow

Package Management

UV (Recommended)

- 10-100x faster than pip
- Rust-powered, built-in resolver

```
uv sync --dev
```

Code Quality

Ruff

- Linting + formatting
- Replaces Black, isort

```
ruff check --fix
```

Makefile

- Task automation
- Common workflows

```
make test lint
```

Thank You ❤️

Start engineering reliable agents with LangChain.